Executions

GIU Curriculum and Academic Innovation Management Platform (GCAIMP) · Milestone 2 · executions divided between the team, same split as Procedures.pdf

From the Milestone 2 document (GIU, Winter 2026, submission 15/11/2026, 11:59 pm):

d) **Executions.sql**: an example execution of every stored procedure and function (**EXEC** for procedures, **SELECT** for functions), using data from your insertions.

• The four SQL files must run in this order without errors: Tables.sql, Insertions.sql, Procedures.sql, Executions.sql.

• It is the students' responsibility to make sure that every stored procedure and function executes correctly and returns the expected output.

• Your procedures and functions must handle any errors or exceptions that may happen (e.g., invalid input, records that do not exist, duplicate records, or violated constraints) and show a clear message instead of failing.

How we do it: write your calls in `executions/<YourName>.sql` on your branch, one execution per procedure or function below, in the same order. Use data from Insertions.sql. Add a second call that triggers an error case where it makes sense. Merge into main and the CI glues it into Executions.sql.

Khalid 23 executions (one per procedure / function)

1 As a System Admin, I should be able to:

1. Retrieve complete information for any academic member using their ID, including their role(s) and access scopes.
   Signature:
   Name: ViewMemberInfo.
   Type: Function.
   Input: @MemberID int.
   Output: Row containing all member details, role(s) and access scopes.

2. Add a new academic member to the system.
   Signature:
   Name: AddAcademicMember.
   Type: Stored procedure.
   Input: @FullName varchar(150), @UniversityIDNo varchar(50), @Email varchar(150), @Phone varchar(50), @AcademicRank varchar(50), @EmploymentStatus varchar(50), @AccountStatus varchar(50), @FacultyID int.
   Output: Confirmation message or new @MemberID int.

3. Update an academic member's contact details.
   Signature:
   Name: UpdateMemberContactInfo.
   Type: Stored procedure.
   Input: @MemberID int, @Email varchar(150), @Phone varchar(50).
   Output: Confirmation message.

4. Activate, suspend, or deactivate a member's account.
   Signature:

Name: UpdateAccountStatus.
Type: Stored procedure.
Input: @MemberID int, @AccountStatus varchar(50).
Output: Confirmation message.

5. Register an existing member as an Academic Instructor.
   Signature:
   Name: AssignInstructorRole.
   Type: Stored procedure.
   Input: @MemberID int, @ResearchArea varchar(150), @OfficeLocation varchar(50), @TeachingLoad int.
   Output: Confirmation message.
6. Register an existing member as a Teaching Assistant under a supervising instructor.
   Signature:
   Name: AssignTeachingAssistantRole.
   Type: Stored procedure.
   Input: @MemberID int, @AssistantLevel varchar(50), @SupervisorID int.
   Output: Confirmation message.
7. Appoint a member as the Dean of a faculty (the faculty can only have one dean at a time).
   Signature:
   Name: AssignDean.
   Type: Stored procedure.
   Input: @MemberID int, @FacultyID int, @ApprovalAuthorityScope varchar(150).
   Output: Confirmation message.
8. Register an existing member as part of University Management.
   Signature:
   Name: AssignManagementRole.
   Type: Stored procedure.
   Input: @MemberID int, @ManagementTitle varchar(150), @DecisionAuthorityLevel varchar(50), @GovernanceScope varchar(150).
   Output: Confirmation message.
9. Register an existing member as a Curriculum Reviewer and record their committee membership.
   Signature:
   Name: AssignReviewerRole.
   Type: Stored procedure.
   Input: @MemberID int, @ReviewDomain varchar(150), @ReviewAuthorityLevel varchar(50), @CommitteeMembership varchar(150).
   Output: Confirmation message.
10. Grant an access scope to a member (e.g., “Submit Proposals”, “Approve Initiatives”).
    Signature:
    Name: GrantAccessScope.
    Type: Stored procedure.
    Input: @MemberID int, @AccessScope varchar(50).
    Output: Confirmation message.
11. Revoke an access scope from a member.
    Signature:
    Name: RevokeAccessScope.
    Type: Stored procedure.

Input: @MemberID int, @AccessScope varchar(50).
Output: Confirmation message.

12. Add a new faculty.
Signature:
Name: AddFaculty.
Type: Stored procedure.
Input: @Name varchar(150), @Description varchar(1000), @EstablishmentDate date, @DeanID int.
Output: Confirmation message or new @FacultyID int.

17. Record the semesters during which a dean is in office.
Signature:
Name: RecordDeanTerm.
Type: Stored procedure.
Input: @DeanID int, @SemesterID int.
Output: Confirmation message.

21. View all platform activity recorded in the audit log within a date range.
Signature:
Name: ViewAuditLog.
Type: Function.
Input: @StartDate date, @EndDate date.
Output: Table showing action type, actor, affected item, timestamp and change summary.

22. Retrieve a summary of academic member distribution across faculties and roles.
Signature:
Name: GetFacultyMemberStats.
Type: Function.
Input: Nothing.
Output: Table showing faculty, role and number of members.

3 As a Teaching Assistant, I should be able to:

2. View my supervising instructor’s details.
Signature:
Name: ViewMySupervisor.
Type: Function.
Input: @TAID int.
Output: Row containing supervisor name, email and office location.

6 As an University Management member, I should be able to:

11. Send an announcement to all members who hold a specific role.
Signature:
Name: BroadcastNotification.
Type: Stored procedure.
Input: @MessageContent varchar(1000), @Urgency varchar(50), @Category varchar(50), @TargetRole varchar(50).
Output: Confirmation message.

12. View the complete audit trail of a proposal, approval, initiative, assessment or piece of evidence.
Signature:
Name: ViewItemAuditTrail.

Type: Function.
Input: @ItemType varchar(20), @ItemID int.
Output: Table showing action type, actor, timestamp and change summary.

7 As an Academic Member, I should be able to:

1. View my own profile.
   Signature:
   Name: ViewMyProfile.
   Type: Function.
   Input: @MemberID int.
   Output: Row containing my details, role(s) and access scopes.

2. View my notifications, optionally showing only unread ones.
   Signature:
   Name: ViewMyNotifications.
   Type: Function.
   Input: @MemberID int, @UnreadOnly bit.
   Output: Table of notifications ordered by timestamp.

3. Count my unread notifications.
   Signature:
   Name: GetUnreadNotificationCount.
   Type: Function.
   Input: @MemberID int.
   Output: Number of unread notifications int.

4. Mark a notification as read.
   Signature:
   Name: MarkNotificationRead.
   Type: Stored procedure.
   Input: @NotificationID int, @MemberID int.
   Output: Confirmation message.

12. Receive a notification when a decision is made on a proposal or initiative I submitted.
    Signature:
    Name: NotifyApprovalDecision.
    Type: Stored procedure.
    Input: @MemberID int, @ItemType varchar(20), @ItemID int, @Decision varchar(50).
    Output: Notification message.

Lamar 24 executions (one per procedure / function)

1 As a System Admin, I should be able to:

13. Add a new program to a faculty.
    Signature:
    Name: AddProgram.
    Type: Stored procedure.
    Input: @Name varchar(150), @DegreeLevel varchar(50), @TotalCreditHours int, @StandardDuration varchar(50), @Description varchar(1000), @FacultyID int.
    Output: Confirmation message or new @ProgramID int.

14. Add a new course as either a Core or an Elective course.
    Signature:
    Name: AddCourse.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @Title varchar(150), @Description varchar(1000), @CreditHours int, @Level varchar(50), @CourseType varchar(20), @DefaultStudySemester int, @RequiredForProgram varchar(150), @ElectivePool varchar(150), @MinSelectionRule varchar(150).
    Output: Confirmation message.

15. Link a course to a prerequisite or co-requisite course.
    Signature:
    Name: AddCourseRequisite.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @RequisiteCode varchar(20), @RequisiteType varchar(50).
    Output: Confirmation message.

2 As an Academic Instructor, I should be able to:

2. Add an intended learning outcome (ILO) to a course.
    Signature:
    Name: AddILO.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @ILOCode varchar(20), @Statement varchar(1000), @CognitiveLevel varchar(50).
    Output: Confirmation message.

3. Update the statement or cognitive level of an existing ILO.
    Signature:
    Name: UpdateILO.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @ILOCode varchar(20), @Statement varchar(1000), @CognitiveLevel varchar(50).
    Output: Confirmation message.

4. Map an ILO of my course to a program learning outcome (PLO).
    Signature:
    Name: MapILOToPLO.
    Type: Stored procedure.

Input: @CourseCode varchar(20), @ILOCode varchar(20), @ProgramID int, @PLOCode varchar(20), @CoverageLevel varchar(50), @IsAssessmentPoint bit.
Output: Confirmation message or new @MappingID int.

8. Submit a curriculum modification proposal, optionally linked to the assessment that motivated it.
   Signature:
   Name: SubmitModificationProposal.
   Type: Stored procedure.
   Input: @ProposerID int, @Title varchar(150), @Rationale varchar(1000), @ModificationType varchar(50), @CurrentStateSummary varchar(1000), @ProposedStateSummary varchar(1000), @CourseCode varchar(20), @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50), @AssessmentID int.
   Output: Confirmation message or new @ProposalID int.

9. Revise a proposal that was returned for revision.
   Signature:
   Name: ReviseModificationProposal.
   Type: Stored procedure.
   Input: @ProposalID int, @Rationale varchar(1000), @ProposedStateSummary varchar(1000).
   Output: Confirmation message.

10. View the full approval history of one of my proposals.
    Signature:
    Name: ViewProposalStatus.
    Type: Function.
    Input: @ProposalID int.
    Output: Table showing decision, decision level, approver, decision date and comments.

11. View the courses of a curriculum version grouped by study semester.
    Signature:
    Name: ViewCurriculumVersion.
    Type: Function.
    Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50).
    Output: Table showing study semester, course code, title, credit hours and whether it is mandatory.

12. Compare two curriculum versions of the same program.
    Signature:
    Name: CompareCurriculumVersions.
    Type: Function.
    Input: @ProgramID int, @OldVersionLabel varchar(20), @OldAcademicYear varchar(50), @NewVersionLabel varchar(20), @NewAcademicYear varchar(50).
    Output: Table of courses that were added, removed or moved to another study semester.

3 As a Teaching Assistant, I should be able to:

3. View the ILOs of a course I support.
   Signature:
   Name: ViewCourseILOs.
   Type: Function.
   Input: @CourseCode varchar(20).
   Output: Table of ILO codes, statements and cognitive levels.

4 As a Curriculum Reviewer, I should be able to:

1. View the full curriculum map of a program (which ILOs map to which PLOs).
   Signature:
   Name: ViewProgramCurriculumMap.
   Type: Function.
   Input: @ProgramID int.
   Output: Table showing PLO, course, ILO, coverage level and assessment point.

2. Identify PLOs that are not covered by any approved mapping.
   Signature:
   Name: GetUncoveredPLOs.
   Type: Function.
   Input: @ProgramID int.
   Output: Table of uncovered PLO codes and statements.

3. View how often each PLO is introduced, reinforced and mastered across courses.
   Signature:
   Name: GetPLOCoverageSummary.
   Type: Function.
   Input: @ProgramID int.
   Output: Table showing PLO, number of courses at each coverage level and number of assessment points.

4. Approve or reject an outcome mapping.
   Signature:
   Name: ReviewOutcomeMapping.
   Type: Stored procedure.
   Input: @MappingID int, @MappingStatus varchar(50).
   Output: Confirmation message.

8. View pending modification proposals for a program.
   Signature:
   Name: ViewPendingProposals.
   Type: Function.
   Input: @ProgramID int.
   Output: Table of proposals with no final decision, with proposer and submission date.

9. Record my review decision on a modification proposal.
   Signature:
   Name: ReviewModificationProposal.
   Type: Stored procedure.
   Input: @ProposalID int, @ReviewerID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

10. View the history of proposals and decisions for a course.
    Signature:
    Name: ViewCourseProposalHistory.
    Type: Function.
    Input: @CourseCode varchar(20).
    Output: Table of proposals with modification type, submission date and latest decision.

5 As a Dean, I should be able to:

2. Approve, reject, or return a modification proposal for revision.
   Signature:

Name: DecideOnProposal.
Type: Stored procedure.
Input: @ProposalID int, @DeanID int, @Decision varchar(50), @Comments varchar(1000).
Output: Confirmation message.

5. Add a program learning outcome (PLO) to a program in my faculty.
   Signature:
   Name: AddPLO.
   Type: Stored procedure.
   Input: @ProgramID int, @PLOCode varchar(20), @Statement varchar(1000), @Category varchar(50).
   Output: Confirmation message.

6. Create a new draft curriculum version, optionally copying the course placements of an existing version.
   Signature:
   Name: CreateCurriculumVersion.
   Type: Stored procedure.
   Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50), @EffectivePeriod varchar(50), @CopyFromVersionLabel varchar(20), @CopyFromAcademicYear varchar(50).
   Output: Confirmation message.

7. Place a course in a curriculum version at a specific study semester.
   Signature:
   Name: PlaceCourseInVersion.
   Type: Stored procedure.
   Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50), @CourseCode varchar(20), @StudySemesterNo int, @IsMandatory bit.
   Output: Confirmation message.

8. Approve a curriculum version, making it Active and archiving the previously active version.
   Signature:
   Name: ApproveCurriculumVersion.
   Type: Stored procedure.
   Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50).
   Output: Confirmation message.

Jomana 23 executions (one per procedure / function)

1 As a System Admin, I should be able to:

16. Add a new academic semester.
    Signature:
    Name: AddSemester.
    Type: Stored procedure.
    Input: @Name varchar(50), @AcademicYear varchar(50), @StartDate date, @EndDate date.
    Output: Confirmation message or new @SemesterID int.

18. Create a course offering (section) in a semester and assign its instructor.
    Signature:
    Name: CreateCourseOffering.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @DeliveryMode varchar(50), @EnrolledCount int, @InstructorID int.
    Output: Confirmation message.

19. Assign a teaching assistant to support a course offering.
    Signature:
    Name: AssignTAToOffering.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @TAID int.
    Output: Confirmation message.

2 As an Academic Instructor, I should be able to:

1. View the course offerings I teach in a given semester.
    Signature:
    Name: ViewMyCourseOfferings.
    Type: Function.
    Input: @InstructorID int, @SemesterID int.
    Output: Table of course offerings with course title, section, delivery mode and enrolled count.

5. Record the result of assessing an ILO in one of my course offerings (whether the target was met is calculated automatically).
    Signature:
    Name: RecordOutcomeAssessment.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @ILOCode varchar(20), @SemesterID int, @SectionID varchar(20), @MappingID int, @Instrument varchar(150), @Description varchar(1000), @TargetThreshold decimal(10,2), @AttainedValue decimal(10,2), @SampleSize int, @AnalysisNotes varchar(1000).
    Output: Confirmation message showing whether the target was met.

6. Add a corrective action to an assessment whose target was not met.
    Signature:
    Name: AddCorrectiveAction.
    Type: Stored procedure.
    Input: @AssessmentID int, @CorrectiveAction varchar(1000).
    Output: Confirmation message.

7. View the outcome assessments in my offerings that did not meet their target.
   Signature:
   Name: ViewUnmetAssessments.
   Type: Function.
   Input: @InstructorID int.
   Output: Table of assessments with ILO, offering, target, attained value and corrective action.

3 As a Teaching Assistant, I should be able to:

1. View the course offerings I support.
   Signature:
   Name: ViewSupportedOfferings.
   Type: Function.
   Input: @TAID int.
   Output: Table of course offerings with course title, semester, section and instructor name.

4. View the outcome assessments recorded for a course offering.
   Signature:
   Name: ViewOfferingAssessments.
   Type: Function.
   Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20).
   Output: Table of assessments with instrument, target, attained value and result.

5. Update the enrolled student count of an offering I support.
   Signature:
   Name: UpdateEnrolledCount.
   Type: Stored procedure.
   Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @EnrolledCount int.
   Output: Confirmation message.

4 As a Curriculum Reviewer, I should be able to:

5. Calculate the attainment rate of a PLO in a semester.
   Signature:
   Name: GetPLOAttainment.
   Type: Function.
   Input: @ProgramID int, @PLOCode varchar(20), @SemesterID int.
   Output: Percentage of linked assessments that met their target decimal(5,2).

6. View all assessments in a program that fell below target in a semester.
   Signature:
   Name: ViewAssessmentsBelowTarget.
   Type: Function.
   Input: @ProgramID int, @SemesterID int.
   Output: Table of assessments with course, ILO, target, attained value and corrective action.

7. Track whether corrective actions on an ILO improved its attainment in later semesters.
   Signature:
   Name: TrackCorrectiveActionEffect.
   Type: Function.
   Input: @CourseCode varchar(20), @ILOCode varchar(20).
   Output: Table showing semester, target, attained value, result and corrective action, ordered by semester.

11. Open a new accreditation review for a program.
    Signature:
    Name: CreateAccreditationReview.
    Type: Stored procedure.
    Input: @ProgramID int, @AccreditingBody varchar(150), @ReviewType varchar(50), @ReviewCycle varchar(50), @NextReviewDue date.
    Output: Confirmation message or new @ReviewID int.

12. Attach all approved outcome mappings of the program to an accreditation review.
    Signature:
    Name: AttachMappingsToReview.
    Type: Stored procedure.
    Input: @ReviewID int.
    Output: Confirmation message showing the number of attached mappings.

13. Attach an outcome assessment to an accreditation review.
    Signature:
    Name: AttachAssessmentToReview.
    Type: Stored procedure.
    Input: @ReviewID int, @AssessmentID int.
    Output: Confirmation message.

14. Attach a piece of initiative evidence to an accreditation review.
    Signature:
    Name: AttachEvidenceToReview.
    Type: Stored procedure.
    Input: @ReviewID int, @EvidenceID int.
    Output: Confirmation message.

15. Update the status and decision of an accreditation review.
    Signature:
    Name: UpdateAccreditationStatus.
    Type: Stored procedure.
    Input: @ReviewID int, @Status varchar(50), @Decision varchar(50), @SubmissionDate date.
    Output: Confirmation message.

16. Generate the full evidence pack of an accreditation review.
    Signature:
    Name: GenerateAccreditationPack.
    Type: Function.
    Input: @ReviewID int.
    Output: Table listing every attached mapping, assessment and evidence item with its details.

17. View accreditation reviews due within a number of days.
    Signature:
    Name: GetUpcomingAccreditations.
    Type: Function.
    Input: @DaysAhead int.
    Output: Table showing program, accrediting body, status and next-review due date.

6 As an University Management member, I should be able to:

7. View the latest accreditation status of every program.

Signature:
Name: GetAccreditationOverview.
Type: Function.
Input: Nothing.
Output: Table showing program, faculty, latest review status, decision and next-review due date.

8. View the outcome attainment trend of a program across semesters.
   Signature:
   Name: GetAttainmentTrend.
   Type: Function.
   Input: @ProgramID int.
   Output: Table showing semester and percentage of assessments that met their target.

7 As an Academic Member, I should be able to:

14. Be notified of upcoming accreditation deadlines for my program.
    Signature:
    Name: NotifyAccreditationDeadline.
    Type: Stored procedure.
    Input: @ProgramID int, @DaysAhead int.
    Output: Notification message.

Abdallah 23 executions (one per procedure / function)

2 As an Academic Instructor, I should be able to:

13. Register a new innovation initiative for a course offering, optionally linked to the assessment that motivated it.
    Signature:
    Name: RegisterInnovationInitiative.
    Type: Stored procedure.
    Input: @SubmitterID int, @FacultyID int, @Title varchar(150), @Description varchar(1000), @Motivation varchar(1000), @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @TargetStartSemesterID int, @ImplementationDeadline date, @AssessmentID int.
    Output: Confirmation message or new @InitiativeID int.

18. Assign academic members or teaching assistants to an initiative with a role (e.g., Lead, Contributor, Supporting TA).
    Signature:
    Name: AssignInitiativeMember.
    Type: Stored procedure.
    Input: @InitiativeID int, @MemberID int, @Role varchar(50).
    Output: Confirmation message.

20. Add an implementation milestone to an initiative (the sequence number is assigned automatically).
    Signature:
    Name: AddMilestone.
    Type: Stored procedure.
    Input: @InitiativeID int, @Title varchar(150), @Description varchar(1000), @DueDate date.
    Output: Confirmation message showing the new @SeqNo int.

21. Mark a milestone as completed.
    Signature:
    Name: CompleteMilestone.
    Type: Stored procedure.
    Input: @InitiativeID int, @SeqNo int, @CompletionDate date.
    Output: Confirmation message.

25. Request approval of an initiative from my faculty's dean.
    Signature:
    Name: RequestInitiativeApproval.
    Type: Stored procedure.
    Input: @InitiativeID int, @MemberID int.
    Output: Confirmation message.

26. Track the progress of an initiative's milestones.
    Signature:
    Name: ViewInitiativeProgress.
    Type: Function.
    Input: @InitiativeID int.
    Output: Table showing each milestone, its due date, completion date, status and whether it is overdue.

27. Calculate the completion percentage of an initiative.
    Signature:

Name: CalculateInitiativeCompletion.
Type: Function.
Input: @InitiativeID int.
Output: Percentage of completed milestones decimal(5,2).

3 As a Teaching Assistant, I should be able to:

6. Update the status of a milestone in an initiative I am assigned to.
   Signature:
   Name: UpdateMilestoneStatus.
   Type: Stored procedure.
   Input: @InitiativeID int, @SeqNo int, @Status varchar(50).
   Output: Confirmation message.

5 As a Dean, I should be able to:

1. View all proposals and initiatives in my faculty awaiting my decision.
   Signature:
   Name: ViewFacultyPendingApprovals.
   Type: Function.
   Input: @DeanID int.
   Output: Table showing item type, item ID, title, submitter and submission date.

3. Approve, reject, or return an innovation initiative for revision.
   Signature:
   Name: DecideOnInitiative.
   Type: Stored procedure.
   Input: @InitiativeID int, @DeanID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

4. Escalate an initiative that needs a university-level decision to University Management.
   Signature:
   Name: EscalateToUniversity.
   Type: Stored procedure.
   Input: @InitiativeID int, @DeanID int, @Comments varchar(1000).
   Output: Confirmation message.

9. View all initiatives in my faculty filtered by status.
   Signature:
   Name: ViewFacultyInitiatives.
   Type: Function.
   Input: @FacultyID int, @Status varchar(50).
   Output: Table of initiatives with course offering, submitter, deadline and status.

10. View all overdue milestones in my faculty.
    Signature:
    Name: ViewOverdueMilestones.
    Type: Function.
    Input: @FacultyID int.
    Output: Table showing initiative, milestone, due date and assigned members.

11. Send overdue alerts to the members assigned to initiatives with overdue milestones.
    Signature:

Name: SendOverdueAlerts.
Type: Stored procedure.
Input: @FacultyID int.
Output: Confirmation message showing the number of notifications sent.

17. View a dashboard summary of my faculty for a semester.
    Signature:
    Name: GetFacultyDashboard.
    Type: Function.
    Input: @FacultyID int, @SemesterID int.
    Output: Row showing number of active initiatives, pending approvals, overdue milestones and percentage of assessments that met their target.

6 As an University Management member, I should be able to:

1. View all items escalated for a university-level decision.
   Signature:
   Name: ViewEscalatedItems.
   Type: Function.
   Input: Nothing.
   Output: Table showing item type, item ID, title, faculty and escalation date.

2. Record a university-level decision on a proposal or initiative.
   Signature:
   Name: MakeUniversityDecision.
   Type: Stored procedure.
   Input: @ItemType varchar(20), @ItemID int, @ManagerID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

3. View a university-wide dashboard for a semester.
   Signature:
   Name: GetUniversityDashboard.
   Type: Function.
   Input: @SemesterID int.
   Output: Table showing each faculty with number of initiatives by status, approval rate and average feedback rating.

4. View initiative statistics by type (AI Tool Integration, Methodology Change, Assessment Redesign).
   Signature:
   Name: GetInitiativeStatsByType.
   Type: Function.
   Input: Nothing.
   Output: Table showing initiative type, status and count.

6. Calculate the average approval turnaround time of a faculty.
   Signature:
   Name: GetApprovalTurnaround.
   Type: Function.
   Input: @FacultyID int.
   Output: Average number of days from submission to final decision int.

9. Rank initiatives by their improvement over baseline for a given metric.

Signature:
Name: RankInitiativesByImpact.
Type: Function.
Input: @MetricType varchar(50).
Output: Table of initiatives ordered by percentage improvement over baseline.

7 As an Academic Member, I should be able to:

5. View the initiatives I submitted or am assigned to.
   Signature:
   Name: ViewMyInitiatives.
   Type: Function.
   Input: @MemberID int.
   Output: Table of initiatives with my role, status and deadline.

13. Be notified when a milestone I am responsible for is overdue.
    Signature:
    Name: NotifyOverdueMilestone.
    Type: Stored procedure.
    Input: @MemberID int, @InitiativeID int, @SeqNo int.
    Output: Notification message.

Rawan 23 executions (one per procedure / function)

1 As a System Admin, I should be able to:

20. Add a new tool to the catalogue as either an AI Tool or a Teaching Methodology.
    Signature:
    Name: AddTool.
    Type: Stored procedure.
    Input: @Name varchar(150), @Description varchar(1000), @Category varchar(50), @Status varchar(50), @ToolType varchar(30), @Provider varchar(150), @ModelVersion varchar(50), @LicenseType varchar(50), @HostingMode varchar(50), @CostModel varchar(50), @OriginatingFramework varchar(150), @ReferenceSource varchar(150), @RequiredResources varchar(1000).
    Output: Confirmation message or new @ToolID int.

2 As an Academic Instructor, I should be able to:

14. Describe the AI tool integration aspect of an initiative.
    Signature:
    Name: AddAIToolIntegration.
    Type: Stored procedure.
    Input: @InitiativeID int, @IntegrationObjective varchar(1000), @ToolCategory varchar(50), @DataPrivacyNotes varchar(1000), @AcademicIntegrityNotes varchar(1000).
    Output: Confirmation message.

15. Describe the methodology change aspect of an initiative.
    Signature:
    Name: AddMethodologyChange.
    Type: Stored procedure.
    Input: @InitiativeID int, @PedagogicalApproach varchar(150), @DeliveryChange varchar(1000), @ReferenceFramework varchar(150).
    Output: Confirmation message.

16. Describe the assessment redesign aspect of an initiative.
    Signature:
    Name: AddAssessmentRedesign.
    Type: Stored procedure.
    Input: @InitiativeID int, @AssessmentTypeAffected varchar(50), @RubricWeightingChanges varchar(1000), @FairnessSafeguards varchar(1000).
    Output: Confirmation message.

17. Link a tool from the catalogue to my initiative.
    Signature:
    Name: AdoptTool.
    Type: Stored procedure.
    Input: @InitiativeID int, @ToolID int.
    Output: Confirmation message.

19. Record a training requirement for an initiative.
    Signature:
    Name: AddTrainingRequirement.
    Type: Stored procedure.

Input: @InitiativeID int, @ToolID int, @Description varchar(1000), @TargetAudience varchar(150), @RequiredByDate date, @IsMandatory bit.
Output: Confirmation message.

22. Upload evidence for an initiative, optionally attached to a specific milestone.
Signature:
Name: UploadEvidence.
Type: Stored procedure.
Input: @InitiativeID int, @MilestoneSeqNo int, @Title varchar(150), @ArtifactType varchar(50), @FilePathOrURL varchar(1000), @Description varchar(1000).
Output: Confirmation message or new @EvidenceID int.

23. Report the measured impact of an initiative for a semester.
Signature:
Name: SubmitOutcomeReport.
Type: Stored procedure.
Input: @InitiativeID int, @SemesterID int, @MetricType varchar(50), @QuantitativeValue decimal(10,2), @BaselineValue decimal(10,2), @QualitativeSummary varchar(1000).
Output: Confirmation message.

24. Record anonymous student feedback about an initiative.
Signature:
Name: RecordStudentFeedback.
Type: Stored procedure.
Input: @InitiativeID int, @SemesterID int, @Rating int, @Comment varchar(1000), @CategorySentiment varchar(50), @CollectionMethod varchar(50).
Output: Confirmation message.

3 As a Teaching Assistant, I should be able to:

7. View all evidence uploaded for an initiative I am assigned to.
Signature:
Name: ViewInitiativeEvidence.
Type: Function.
Input: @InitiativeID int.
Output: Table of evidence with title, artifact type, milestone and upload date.

5 As a Dean, I should be able to:

12. Schedule a training session (online or in person) for a tool.
Signature:
Name: ScheduleTrainingSession.
Type: Stored procedure.
Input: @ToolID int, @Title varchar(150), @Topic varchar(150), @Description varchar(1000), @SessionDate date, @DurationMinutes int, @Trainer varchar(150), @Capacity int, @Mode varchar(20), @Platform varchar(50), @MeetingLink varchar(1000), @RecordingAvailable bit, @Venue varchar(150), @Campus varchar(50), @RequiredEquipment varchar(1000).
Output: Confirmation message or new @SessionID int.

13. View the attendance list of a training session.
Signature:
Name: ViewTrainingAttendance.
Type: Function.

Input: @SessionID int.
Output: Table of attending members with name, rank and email.

14. Check training compliance for the initiatives in my faculty.
    Signature:
    Name: GetFacultyTrainingCompliance.
    Type: Function.
    Input: @FacultyID int.
    Output: Table showing initiative, training requirement, number of assigned members and number who attended matching training.

15. View the measured impact of an initiative compared to its baseline.
    Signature:
    Name: GetInitiativeImpact.
    Type: Function.
    Input: @InitiativeID int.
    Output: Table showing semester, metric type, baseline, value and percentage change.

16. Calculate the average student feedback rating of an initiative in a semester.
    Signature:
    Name: GetAverageFeedbackRating.
    Type: Function.
    Input: @InitiativeID int, @SemesterID int.
    Output: Average rating decimal(3,2).

6 As an University Management member, I should be able to:

5. View how widely each tool has been adopted.
    Signature:
    Name: GetToolAdoptionReport.
    Type: Function.
    Input: Nothing.
    Output: Table showing tool, number of initiatives, number of faculties and number of trained members.

10. Change the status of a tool university-wide (e.g., Approved, Under Review, Suspended).
    Signature:
    Name: UpdateToolStatus.
    Type: Stored procedure.
    Input: @ToolID int, @Status varchar(50).
    Output: Confirmation message.

7 As an Academic Member, I should be able to:

6. View upcoming training sessions for a tool.
    Signature:
    Name: ViewAvailableTrainingSessions.
    Type: Function.
    Input: @ToolID int.
    Output: Table of sessions with date, mode, trainer and remaining seats.

7. Check the number of remaining seats in a training session.
    Signature:
    Name: GetRemainingSeats.

Type: Function.
Input: @SessionID int.
Output: Number of remaining seats int.

8. Register for a training session (only if seats are available).
   Signature:
   Name: RegisterForTraining.
   Type: Stored procedure.
   Input: @SessionID int, @MemberID int.
   Output: Confirmation message.

9. Cancel my registration in a training session before it takes place.
   Signature:
   Name: CancelTrainingRegistration.
   Type: Stored procedure.
   Input: @SessionID int, @MemberID int.
   Output: Confirmation message.

10. View my training history.
    Signature:
    Name: ViewMyTrainingHistory.
    Type: Function.
    Input: @MemberID int.
    Output: Table of attended sessions with tool, date and mode.

11. View the mandatory training I still need for the initiatives I am assigned to.
    Signature:
    Name: ViewMyPendingTraining.
    Type: Function.
    Input: @MemberID int.
    Output: Table showing initiative, requirement, tool and required-by date.